

Foreman

Supervise everything your agents build — and prove it's healthy.

The control plane for a portfolio of software built and maintained by coding agents: it schedules the recurring reviews, collects a verdict from every run, and surfaces what needs you. Schedules and reads — the prompts do the work.

M1–M8 shipped · 304 tests · ~86% coverage

receipts in git · green/amber/red verdicts · declared-vs-effective drift · cloud / local / session tiers · one board

The shift

"The bottleneck moved from writing code to supervising many autonomous streams of it."

- ▷ Coding agents made it cheap to create and maintain far more software per person.
- ▷ One operator now runs 10–30 agent-maintained projects at once.
- ▷ Nothing exists to keep them all healthy without babysitting each one.



The problem

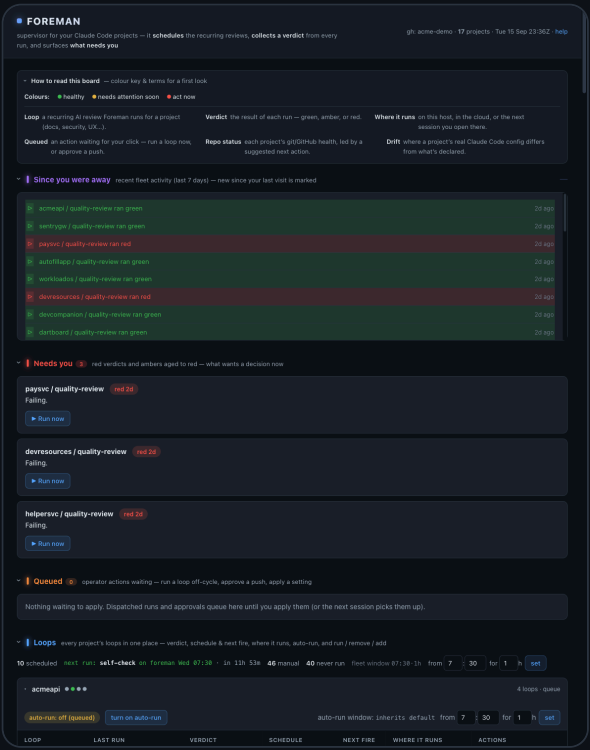
- ▷ Is each of 20 repos still documented? secure? prod-ready? on budget?
- ▷ CI only fires on commits — and only runs deterministic checks.
- ▷ The reviews that matter (architecture, UX, pen-test, prod-readiness) are open-ended judgment no linter can make.
- ▷ Doing it by hand, per repo, every week — doesn't scale.

What Foreman is

"A control plane: it schedules AI reviews across every project, reduces each to a durable green/amber/red verdict, and shows the fleet on one board — with a git-committed receipt behind every verdict."

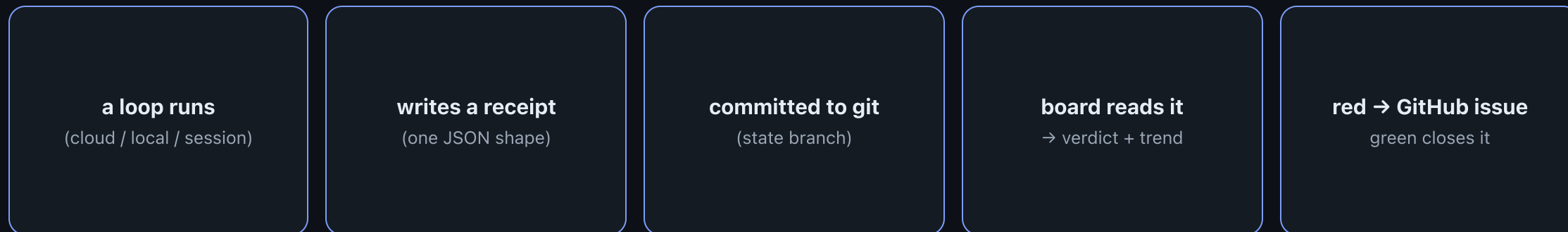
- ▷ A control repo, a contract, and a small index — not a daemon, not an agent runtime.
- ▷ Foreman never does the work; the cadence prompts do.

One glance: catch up on the whole fleet



Every section folds to a one-line peek and opens on signal, so the board is a scannable digest. “How to read this board” decodes the colour language · fleet catch-up (“Since you were away”) and triage (“Needs you” · “Queued” one-click actions) lead · “Recently worked on” is a searchable per-project prompt history.

How it works — every run leaves a receipt in git



A run with no receipt is red by absence — silent scheduler failures become visible. The SQLite index is just a cache, fully rebuildable from the receipts.

The loop library

FOREMAN

superior for your Claude Code projects — it schedules the recurring reviews, collects a verdict from every tool, and surfaces what needs you

gh:acme-demo · 17 projects · Tue 16 Sep 22 16:2 · help

LOOP	KIND	ENABLED FOR	ENABLE FOR...	EDIT
docs-sync Documentation and user guide diff veritas: open PR, CI fix, + foreman/docs-sync capex: 0:45m escalates on red	built-in	acmeapi, acmeweb, audiosvc, autoflags, classtot, dashboard, devcompanion, devresources, flingovic, helpervnc, inventoryapp, jumpstart, payroll, payroll, sentrygo, workbooks	foreman enable	% edit
quality-review Code quality and review diff veritas: no PR, CI fix, + foreman/quality-review capex: 0:45m escalates on red	built-in	acmeapi, acmeweb, audiosvc, autoflags, classtot, dashboard, devcompanion, devresources, foreman, helpervnc, inventoryapp, jumpstart, payroll, payroll, sentrygo, workbooks	flingovic enable	% edit
beta-readiness Beta readiness blockers veritas: no PR, CI fix, + foreman/beta-readiness capex: 0:45m escalates on red	built-in	acmeapi	sentrygo enable	% edit
harness-refresh Claude Code harness diff veritas: no PR, CI fix, + foreman/harness-refresh capex: 0:20m	built-in	sentrygo	acmeapi enable	% edit
arch-review Architecture and boundaries review veritas: no PR, CI fix, + foreman/arch-review capex: 0:45m escalates on red	built-in	acmeapi	sentrygo enable	% edit
security-review Security review veritas: no PR, CI fix, + foreman/security-review capex: 0:45m escalates on red	built-in	acmeweb, audiosvc, autoflags, classtot, dashboard, devcompanion, devresources, foreman, helpervnc, inventoryapp, jumpstart, payroll, workbooks	acmeapi enable	% edit
soc2-readiness SOC 2 readiness read-only capex: 0:20m escalates on red	built-in	foreman	acmeapi enable	% edit
ui-ux-review UI/UX review read-only capex: 0:30m escalates on red	built-in	foreman	acmeapi enable	% edit
intuitive-ux First-use intuitiveness review read-only capex: 0:30m escalates on red	built-in	foreman	acmeapi enable	% edit
perf-review Performance and scalability review veritas: no PR, CI fix, + foreman/perf-review capex: 0:45m escalates on red	built-in	foreman	acmeapi enable	% edit
prod-readiness Production readiness review	built-in	none	acmeapi enable	% customize

docs · quality · security · soc2 · arch · perf · prod-readiness · pen-test · test · UI/UX — one-click per project, or customize a built-in / add your own.

Every project's loops, in one panel

FOREMAN
supervisor for your Claude Code projects – it **schedules** the recurring reviews, **collects a verdict** from every run, and surfaces **what needs you**

gh:acme/demo 17 projects · Tue 16 Sep 23 06:2 · help

auto-run: off (paused) · turn on auto-run auto-run window: liber!ts: default: from 7 30 for 1 h set

LOOP	LAST RUN	VERDICT	SCHEDULE	NEXT FIRE	WHERE IT RUNS	ACTIONS
docs-sync	never	state	auto-run required	manual	this host	▶ Run ✕ remove
quality-review	2026-09-13T23:34:28Z	green 2d	auto-run required	manual	default (cloud)	▶ Run ✕ remove
beta-readiness	never	state	auto-run required	manual	default (cloud)	▶ Run ✕ remove
arch-review	never	state	auto-run required	manual	default (cloud)	▶ Run ✕ remove

+ Add a loop to this project

3 loops · quiet

auto-run: off (paused) · turn on auto-run auto-run window: liber!ts: default: from 7 30 for 1 h set

LOOP	LAST RUN	VERDICT	SCHEDULE	NEXT FIRE	WHERE IT RUNS	ACTIONS
docs-sync	never	state	auto-run required	manual	this host	▶ Run ✕ remove
quality-review	2026-09-13T23:34:28Z	green 2d	auto-run required	manual	default (cloud)	▶ Run ✕ remove
harness-refresh	never	state	auto-run required	manual	default (this host)	▶ Run ✕ remove

+ Add a loop to this project

2 loops · quiet

auto-run: off (paused) · turn on auto-run auto-run window: liber!ts: default: from 7 30 for 1 h set

LOOP	LAST RUN	VERDICT	SCHEDULE	NEXT FIRE	WHERE IT RUNS	ACTIONS
docs-sync	never	state	auto-run required	manual	default (cloud)	▶ Run ✕ remove
quality-review	2026-09-13T23:34:28Z	red 2d	auto-run required	manual	default (cloud)	▶ Run ✕ remove

+ Add a loop to this project

3 loops · quiet

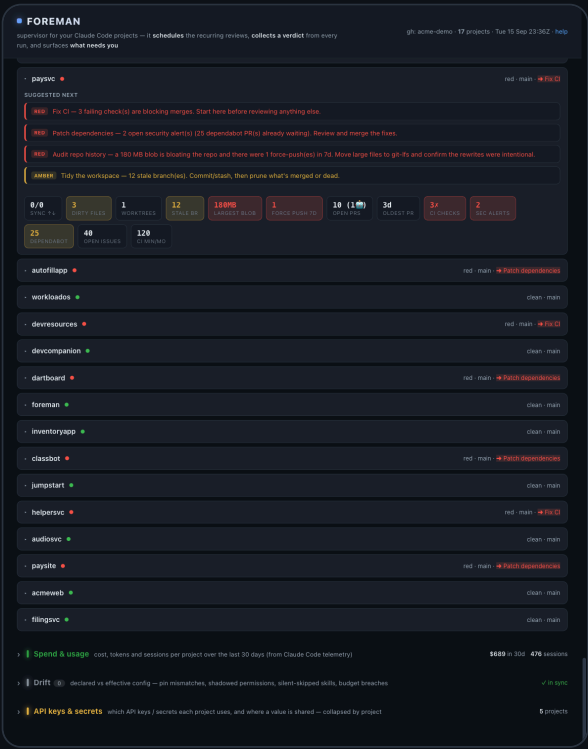
auto-run: off (paused) · turn on auto-run auto-run window: liber!ts: default: from 7 30 for 1 h set

LOOP	LAST RUN	VERDICT	SCHEDULE	NEXT FIRE	WHERE IT RUNS	ACTIONS
docs-sync	never	state	auto-run required	manual	default (cloud)	▶ Run ✕ remove
quality-review	2026-09-13T23:34:28Z	green 2d	auto-run required	manual	default (cloud)	▶ Run ✕ remove
security-review	never	state	auto-run required	manual	default (cloud)	▶ Run ✕ remove

+ Add a loop to this project

verdict + trend sparkline · flexible schedule (“every N days”, weekdays, weekly...) & next run · where it runs · run / remove / add.

Fleet health — with a recommendation, not just numbers



Each repo leads with “Suggested next” — the raw git/GitHub stats synthesized into a short, prioritized list of what to act on (red → amber). Related signals fold into one action; the top one shows in the collapsed row. Vital-sign tiles and a cross-project roll-up sit below as detail.

The loop closes itself

"amber ages to red → red opens a GitHub issue → a green run closes it, linking the clearing receipt."

- ▷ Every verdict is a schema-validated JSON receipt committed to git.
- ▷ A tamper-evident, versioned record of what was reviewed, when, and how it turned out.

Why it's different — the moats

CONTRACT · Receipt-as-contract, index-as-cache

Every verdict is a git-committed JSON receipt; the database is a throwaway cache. Inspectable, versioned, portable — not a stateful black box.

DRIFT · Declared $\neq$ effective

Foreman probes what's actually in effect (`claude -p`) and reports config drift — pins, shadowed permissions, silently-skipped skills. No comparable tool does this.

METERED · Portfolio + time, metered

Cloud/local/session tiers, per-run budgets, quota guard, spend by project — agent runs as a metered resource across a fleet.

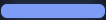
SUPERVISOR · A supervisor, not a swarm

"Never does the work" keeps it composable and safe — it orchestrates when agents run and reads what they concluded.

The wedge: continuous assurance

"The security / soc2 / pen-test loops + the git-committed receipt history are, together, audit evidence that every project was reviewed on a schedule."

- ▷ Every verdict and its clearing, versioned in git.
- ▷ Not a nice-to-have — audit evidence with a buyer and a budget.



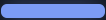
Where it sits

- ▷ Not CI/CD — cadence-driven, not commit-driven; open-ended AI reviews, not scripts.
- ▷ Not a code-quality SaaS — LLM judgment (arch, UX, pen-test), portfolio-wide.
- ▷ Not a coding agent — the layer above; it never does the work.
- ▷ Not an agent-swarm framework — a standing supervisor across projects and time.
- ▷ Uniquely: the config-drift probe, and the rebuildable git-native receipt trail.



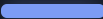
Who it's for

- ▷ The operator — solo builder, studio, or platform team — running many agent-maintained projects.
- ▷ Who needs portfolio-level assurance: is every project still documented, secure, prod-ready, on budget — and can I prove it?



Status & honest limits

- ▷ Shipped: M1–M8 + secrets store + full operator surface. 304 tests, ~86% coverage.
- ▷ Live as a local dashboard + a token-gated read-only hosted board.
- ▷ Deeply wired to Claude Code today (the drift probe is the ecosystem-specific moat).
- ▷ Verdicts are LLM judgments — mitigated by an independent verifier pass.
- ▷ Single-operator today, not yet multi-tenant SaaS; cloud-tier ops partly host-bound.



The bottom line

"A supervisor you can trust and reconstruct — not an agent swarm."

▷ [README.md](#) · [docs/ONE-PAGER.md](#) · [docs/COMPARISON.md](#)